

# Language Agent Tree Search

## Practice Problems

---

These practice problems accompany the LATS lecture. Work through the video and the slides before starting. This problem set is intended for advanced undergraduates or first-year graduate students, and the whole set should take **30 minutes to an hour**.

- A calculator is useful for Part 2. Part 3 needs a web browser; Parts 1 and 2 do not.
- The reference sheet on the last page carries all important technical details you may need.
- Round to three decimal places. Small differences from the numbers shown in the visualizer are rounding, not mistakes.
- Questions marked *optional* are there if you want to go further.

### Part 1: Concepts

Mark one option per question.

#### Question A.

In LATS, when is the value  $V(s)$  of a node computed?

- ☐ (a) Before the action is taken, from the text of the proposed action alone.
- ☐ (b) After the action has been executed in the environment and the observation has come back.
- ☐ (c) Only at terminal nodes, once the environment has returned a reward.
- ☐ (d) During backpropagation, from the rewards of the node's descendants.

#### Question B.

A node  $s$  has  $V(s) = 0.40$  and  $N(s) = 3$ . Its parent  $p$  has been visited  $N(p) = 8$  times. The exploration weight is  $w = 1$ . Using  $\ln 8 \approx 2.0794$ , what is  $UCT(s)$ ?

- ☐ (a) 0.77
- ☐ (b) 0.83
- ☐ (c) 1.09
- ☐ (d) 1.23

**Question C.**

The LATS paper ablates three components on HotPotQA, where the full system scores 0.63 exact match. Which single change costs the most accuracy?

- ☐ (a) Removing reflection.
- ☐ (b) Replacing Monte Carlo Tree Search with plain depth-first search.
- ☐ (c) Removing the language model's value estimate.
- ☐ (d) All three cost roughly the same amount.

**Question D.**

In the `openai_multihop-qa` trace, the agent is asked at which venue, and in which year, the paper that introduced the algorithm LATS adapts was presented. Its first trajectory answers "*European Conference on Machine Learning (ECML), 2006*" and receives reward 0. Why is that answer wrong, but tempting?

- ☐ (a) ECML 2006 is where UCT was introduced. The question asks about Monte Carlo Tree Search itself, which Coulom introduced at Computers and Games in 2006.
- ☐ (b) The venue is right, but the year is wrong.
- ☐ (c) ECML did not hold a conference in 2006.
- ☐ (d) The retrieval tool returned no documents, so the agent guessed.

**Question E.**

Two traces solve the same Game of 24 puzzle. In `openai_game-of-24`, a real language model finishes in one iteration having expanded 16 nodes. In `mock_game-of-24`, the offline policy needs five iterations and 45 nodes. What is the right conclusion?

- ☐ (a) The real model is running a better search algorithm.
- ☐ (b) When the policy already proposes a winning action, the search adds cost and buys nothing.
- ☐ (c) Tree search is never worth its cost on reasoning tasks.
- ☐ (d) The offline policy is implemented incorrectly.

## Part 2: Tracing one round of selection

In the `openai_multihop-qa` trace, the first iteration expands three children of the root, rolls one trajectory out to a terminal state, receives reward 0, and backpropagates it. The second iteration then begins with selection at the root. At that moment the three children are:

- **Node 1**    `search[LATS Language Agent Tree Search adapts algorithm venue year]`
- **Node 2**    `search["Language Agent Tree Search" paper]`
- **Node 3**    `search[UCT algorithm introduced venue year ECML 2006]`

The root has been visited  $N(p) = 2$  times and the exploration weight is  $w = 1$ . Use  $\ln 2 \approx 0.6931$ .

**Question A.** Complete the table. Give three decimal places.

| Node | $V(s)$ | $N(s)$ | Exploration bonus<br>$w \sqrt{\ln N(p) / N(s)}$ | UCT( $s$ ) |
|------|--------|--------|-------------------------------------------------|------------|
| 1    | 0.2833 | 2      |                                                 |            |
| 2    | 0.5417 | 1      |                                                 |            |
| 3    | 0.3917 | 1      |                                                 |            |

**Question B.** Which node does selection take?

**Question C.** Immediately after evaluation in iteration 1, node 1 had the *highest* value of the three:  $V = 0.5667$ , against 0.5417 for node 2. In one or two sentences, explain what changed between then and now.

**Question D (optional).** Suppose backpropagation had lowered node 1's value to 0.2833 but left its visit count at 1. Would selection still take node 2? Show the comparison.

### Part 3: Reading an ablation from the visualizer

Open the visualizer at <https://ryquo-labs.github.io/LATS-Visualizer/>. Use the trace picker in the top right to load `mock_game-of-24`, then `mock_game-of-24_greedy`. The search configuration is shown as labelled chips in the header. Stepping the timeline to its *last* step shows the final result, including the iteration and node counts.

**Question A.** Exactly one field in the search configuration differs between the two traces. Name the field and give its value in each trace.

Field: \_\_\_\_\_

Value in `mock_game-of-24`: \_\_\_\_\_

Value in `mock_game-of-24_greedy`: \_\_\_\_\_

**Question B.** Complete the table from the final step of each run.

| Trace                               | Iterations run | Nodes | Solved? |
|-------------------------------------|----------------|-------|---------|
| <code>mock_game-of-24</code>        |                |       |         |
| <code>mock_game-of-24_greedy</code> |                |       |         |

**Question C.** The greedy run expanded *more* nodes than the full search and still failed to solve the puzzle. Explain why, in terms of the two terms of the UCT formula. Two or three sentences.

**Question D (optional).** Load `openai_game-of-24_hard` and then `openai_game-of-24_hard_wide`. Again exactly one search parameter differs. What does this pair show about a limit of tree search that no number of extra iterations can fix?

## Reference sheet

### The three equations

|                        |                                                                                                       |
|------------------------|-------------------------------------------------------------------------------------------------------|
| <b>Selection (UCT)</b> | $\text{UCT}(s) = V(s) + w \sqrt{\frac{\ln N(p)}{N(s)}}$                                               |
| <b>Value</b>           | $V(s) = \lambda \cdot \text{LM}(s) + (1 - \lambda) \cdot \text{SC}(s)$                                |
| <b>Backup</b>          | $N(s) \leftarrow N(s) + 1, \quad V(s) \leftarrow \frac{V_{\text{old}}(s) \cdot (N(s) - 1) + r}{N(s)}$ |

### Notation

|                |                                                                                                 |
|----------------|-------------------------------------------------------------------------------------------------|
| $s$            | A node, identified with the transcript that reaches it.                                         |
| $p$            | The parent of $s$ .                                                                             |
| $V(s)$         | The current value estimate of $s$ .                                                             |
| $N(s)$         | The number of visits to $s$ . Initialized to 1, not 0.                                          |
| $w$            | The exploration weight. Larger $w$ favours children with fewer visits.                          |
| $\lambda$      | The mixing weight between the two sources of value.                                             |
| $\text{LM}(s)$ | The model's own score for the transcript at $s$ .                                               |
| $\text{SC}(s)$ | Self-consistency: of the $n$ actions sampled at the parent of $s$ , the fraction matching $s$ . |
| $r$            | The reward the environment returns at a terminal state.                                         |

### The six operations

|                        |                                                                                           |
|------------------------|-------------------------------------------------------------------------------------------|
| <b>Selection</b>       | Walk down from the root, taking the highest-UCT child, until reaching an unexpanded node. |
| <b>Expansion</b>       | Sample $n$ candidate actions and execute each one in the environment.                     |
| <b>Evaluation</b>      | Score each new node with $V(s)$ , after the environment has replied.                      |
| <b>Simulation</b>      | Descend greedily by value, in the real environment, to a terminal state.                  |
| <b>Backpropagation</b> | Send $r$ up the path to the root, updating $V$ and $N$ at every ancestor.                 |
| <b>Reflection</b>      | On failure, write a note about why, and add it to the context of later expansions.        |

### Defaults used in the paper

- $n = 5$  samples per expansion,  $w = 1$ , and  $N$  initialized to 1.
- $\lambda = 0.5$  for question answering and Game of 24;  $\lambda = 0.8$  for programming and web tasks.
- In the programming setting, simulation is skipped, because every node is already a complete program. The reward is the fraction of tests that pass.

**Useful values**     $\ln 2 \approx 0.6931$      $\ln 3 \approx 1.0986$      $\ln 8 \approx 2.0794$

**Using the visualizer**    <https://ryquo-labs.github.io/LATS-Visualizer/>

Source code: <https://github.com/Ryquo-Labs/LATS-Visualizer>

- The *trace picker* in the top right switches between runs. The *header* shows the task, the configuration as labelled chips ( $n$ ,  $w$ ,  $\lambda$ , depth), the best reward, and the policy.
- The *timeline* steps through the run one operation at a time. Its last step reports the iterations run, the node count, and whether the task was solved.
- Clicking a node shows its action, the observation the environment returned, and its current  $V$  and  $N$ .